SOLID Design Principles

SOLID is a set of five object-oriented design principles that help make software easier to maintain, extend, test, and understand.

S - Single Responsibility Principle (SRP)

Each class should have one responsibility and one reason to change. Example: UserService handles users while EmailService sends emails.

O - Open/Closed Principle (OCP)

Software entities should be open for extension but closed for modification. Add new behavior by creating new classes instead of editing existing ones. Factory Pattern is a common example.

L - Liskov Substitution Principle (LSP)

Any subclass should be usable wherever its parent class is expected without breaking the program. If Bird has fly(), Penguin should not inherit from it if it cannot fly.

I - Interface Segregation Principle (ISP)

Clients should not depend on methods they do not use. Prefer several small interfaces over one large interface.

D - Dependency Inversion Principle (DIP)

High-level modules should depend on abstractions (interfaces), not concrete implementations. Use interfaces and dependency injection.

SOLID with Factory Pattern

Your Pizza example follows OCP because adding SeaFoodPizza only requires creating a new class and updating the factory. It also uses DIP because the services depend on the PizzaService interface instead of concrete pizza classes.

Interview Tips

1. Know all five principles by name.
2. Explain the problem before the solution.
3. Give a Java example for each principle.
4. Mention design patterns that support SOLID, such as Factory, Strategy, and Dependency Injection.